

Topic 10: AVL Trees and Balancing

10.1 The Problem with Unbalanced BSTs

- Inserting sorted data (1,2,3,...) into a BST creates a right-skewed tree -- $O(n)$ all operations
- Need self-balancing trees to guarantee $O(\log n)$ height
- AVL tree (Adelson-Velsky and Landis, 1962): first self-balancing BST
- Balance Factor of a node = $\text{height}(\text{left subtree}) - \text{height}(\text{right subtree})$
- AVL property: balance factor must be -1, 0, or +1 for every node

10.2 AVL Rotations

- When an insertion violates the AVL property, rotations restore balance
- Right Rotation (LL case): unbalanced right-right -- rotate left-heavy subtree clockwise
- Left Rotation (RR case): rotate right-heavy subtree counter-clockwise
- Left-Right Rotation (LR case): left rotation on left child, then right rotation on node
- Right-Left Rotation (RL case): right rotation on right child, then left rotation on node

```
TreeNode* rotateRight(TreeNode* y) {
    TreeNode* x = y->left;
    TreeNode* T2 = x->right;
    x->right = y;
    y->left = T2;
    // update heights ...
    return x;
}

TreeNode* rotateLeft(TreeNode* x) {
    TreeNode* y = x->right;
    TreeNode* T2 = y->left;
    y->left = x;
    x->right = T2;
    return y;
}
```

10.3 AVL Insert Overview

- Perform standard BST insert
- Update height of current node: $1 + \max(\text{height}(\text{left}), \text{height}(\text{right}))$
- Compute balance factor
- Apply appropriate rotation if balance factor falls outside $[-1, 1]$

- Guaranteed $O(\log n)$ height -- at most $O(\log n)$ rotations per insertion

10.4 Other Balanced BSTs

- Red-Black Tree: looser balance guarantee, faster insertions -- used in `std::map` and `std::set`
 - B-Tree: self-balancing for disk storage -- used in databases and file systems
 - Splay Tree: recently accessed elements are faster to access next time
-